



Chapter	Topic	Key Concepts
1	Foundations of dbt in the Modern Data Stack	ELT, dbt myths, logic vs compute
2	The dbt Ecosystem: Core, Cloud & Canvas	Feature comparison, when to use each
3	Environment Setup & Modern Tooling	UV vs pip, Python 3.12, VS Code setup
4	Project Initialization & dbt_project.yml	profiles.yml, dbt_project.yml, dbt debug
5	Medallion Architecture (Models)	Bronze/Silver/Gold, source(), ref()
6	dbt Seeds: Supplemental Static Data	CSV seeds, configuration, use cases
7	Advanced Templating: Jinja & Macros	Variables, macros, loop.last pattern
8	Data Quality Framework (Testing)	Generic tests, singular tests, test strategy
9	Capturing History: Snapshots & SCD Type 2	Dedup strategy, snapshot config
10	Documentation & Lineage (NEW)	dbt docs generate/serve, DAG exploration
11	Orchestration, CI/CD & Deployment	CLI commands, node selection, schema macro
12	Quick Reference Card (NEW)	Commands, patterns, cheat sheet

- Medallion Architecture
- Seeds, Macros & Jinja
- Snapshots & SCD Type 2
- CI/CD & Production Deployment
- dbt Project Configuration
- Testing & Data Quality
- Documentation & Lineage
- Advanced Patterns

1

Foundations of dbt in the Modern Data Stack

dbt (Data Build Tool) serves as the transformation layer — the **T** in ELT (Extract, Load, Transform). It empowers analytics engineers to write modular SQL while bringing software engineering best practices to the data warehouse: version control, automated testing, and living documentation.

What dbt IS and IS NOT

Capability	Reality	What to Use Instead
Orchestration	dbt does NOT schedule or trigger pipelines	Airflow, Azure Data Factory, Lakeflow
Data Storage	dbt does NOT store data — it only transforms	Snowflake, Databricks, BigQuery
Data Extraction	dbt cannot pull data from APIs or sources	Fivetran, Airbyte, custom Python
Transformation ■	dbt DOES compile Jinja-SQL → raw SQL	dbt Core / Cloud
Testing ■	dbt DOES run data quality assertions	Generic + Singular tests
Documentation ■	dbt DOES generate lineage docs	dbt docs generate + serve

The Logic vs. Compute Relationship

dbt provides the **templating logic** (Jinja + SQL). Your cloud platform provides the **compute power**. When you run a dbt model, dbt compiles the Jinja-SQL into raw SQL and sends it to Databricks or Snowflake for execution.

dbt's Job	Platform's Job
Parse Jinja templates	Execute compiled SQL
Resolve ref() / source() calls	Allocate cluster / warehouse compute
Generate manifest.json	Store result tables / views
Run tests & assertions	Scan data to evaluate test conditions

2

The dbt Ecosystem: Core, Cloud & Canvas

Feature	dbt Core	dbt Cloud	dbt Canvas
Cost	Free (Open Source)	Paid (Managed SaaS)	Included w/ Cloud
Interface	Command Line (CLI)	Managed UI / IDE	Drag-and-Drop GUI
Management	Self-managed	dbt Labs managed	Managed in Cloud
Job Scheduling	■ External tool needed	■ Built-in scheduler	■ via Cloud
Git Integration	Manual (any Git host)	Managed Git	via Cloud
IDE / Editor	VS Code + extensions	Cloud IDE	Visual canvas
Best For	Developers / CI-CD	Teams / Enterprises	Low-code users

dbt Canvas — When to Use It

dbt Canvas is a visual drag-and-drop interface built into dbt Cloud. It bridges the gap between SQL-fluent analytics engineers and business stakeholders who prefer a no-code workflow.

Use dbt Canvas when...	Stick with dbt Core when...
Business users want to explore the DAG visually	You need full Git control and custom macros
Teams need a low-code onramp for basic transforms	Complex Jinja logic or custom materializations required
You want BI-style collaboration without a separate tool	CI/CD pipelines and automated testing are critical

3

Environment Setup & Modern Tooling

UV vs. pip — Why the Modern Toolchain Matters

■ NEW IN THIS EDITION

This comparison is NEW in the enhanced edition — the original PDF mentioned UV but did not explain the rationale.

■ pip (Traditional)

- Slow dependency resolution
- Global env conflicts possible
 - No lock file by default
- Needs venv + pip separately

VS

■ UV (Modern)

- 10–100x faster than pip
- Automatic lock file (uv.lock)
 - Isolated envs built-in
- Single tool: init + add + sync

UV is a next-generation Python package manager written in Rust. For dbt projects it matters because dbt has a large dependency tree — UV resolves it in seconds instead of minutes, and the generated uv.lock file guarantees every team member uses identical versions.

Step-by-Step Setup

1	Install VS Code	Industry-standard editor. Add extensions: dbt Power User, Python, GitLens.
2	Install Git	Required for version control. Verify: <code>git --version</code>
3	Install Python 3.12	dbt is INCOMPATIBLE with Python 3.13. Use exactly 3.12.x
4	Install UV	Run: <code>pip install uv</code> (one-time global install)
5	Bootstrap project	<code>uv init</code> → <code>uv add dbt-core dbt-databricks</code> → <code>uv sync</code>
6	Configure VS Code	Associate <code>.sql</code> → Jinja SQL, <code>.yml</code> → Jinja YAML in dbt Power User

Bootstrap Commands

BASH

```
# Verify base tools
python --version  # Must be 3.12.x – NOT 3.13
git --version
uv --version

# Initialize a modern Python environment
uv init
uv add dbt-core dbt-databricks
uv sync

# Verify dbt is ready
dbt --version
```

■■ WARNING

Python 3.13 is NOT compatible with dbt Core as of this writing. Always use Python 3.12.x. UV makes this easy — you can pin: `uv p`

4

Project Initialization & Configuration Files

■ NEW IN THIS EDITION

dbt_project.yml is a foundational file that was MISSING from the original PDF. This chapter covers it fully.

dbt init & Directory Structure

BASH

```
dbt init my_project
# Creates:
# my_project/
# ████ dbt_project.yml      ← Project-level config (NEW)
# ████ profiles.yml        ← Connection config (local only)
# ████ models/
#   ████ bronze/
#   ████ silver/
#   ████ gold/
# ████ seeds/              ← CSV reference data
# ████ tests/              ← Singular tests
# ████ macros/             ← Reusable Jinja functions
# ████ snapshots/          ← SCD Type 2 configs
# ████ target/             ← Compiled SQL output (gitignore!)
```

dbt_project.yml — The Project Configuration File

This is the **most important configuration file** in any dbt project. It controls model materializations, custom schema assignments, variable defaults, and seed configurations. Every dbt project must have this file.

YAML

```
# dbt_project.yml
name: 'my_dbt_project'
version: '1.0.0'
config-version: 2

# Which profile in profiles.yml to use
profile: 'my_dbt_project'

# Model directory paths
model-paths: ["models"]
seed-paths:  ["seeds"]
test-paths:  ["tests"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

# Default materialization strategy
models:
  my_dbt_project:
    # Bronze = views (cheap, raw pass-through)
    bronze:
      +materialized: view
      +schema: bronze

    # Silver = tables (persisted, transformed)
    silver:
      +materialized: table
      +schema: silver

    # Gold = tables (business-ready aggregations)
    gold:
      +materialized: table
      +schema: gold

# Seed configuration
seeds:
  my_dbt_project:
    +schema: reference_data
    +quote_columns: false

# Project-level variables
vars:
  start_date: '2024-01-01'
  env: 'dev'
```

profiles.yml — Connection Configuration

■ ■ NOTE

profiles.yml lives in ~/.dbt/ for local dev. Never commit tokens to Git! Use environment variables: token: "{{ env_var('DBT_TOKEN') }}"

YAML

```
# ~/.dbt/profiles.yml
my_dbt_project:
  outputs:
    dev:
      type: databricks
      catalog: dbt_tutorial_dev
      host: "{{ env_var('DBT_HOST') }}"
      http_path: "{{ env_var('DBT_HTTP_PATH') }}"
      token: "{{ env_var('DBT_TOKEN') }}"
      schema: default
      threads: 4

    prod:
      type: databricks
      catalog: dbt_tutorial_prod
      host: "{{ env_var('DBT_HOST_PROD') }}"
      http_path: "{{ env_var('DBT_HTTP_PATH_PROD') }}"
      token: "{{ env_var('DBT_TOKEN_PROD') }}"
      schema: default
      threads: 8

  target: dev  # Default target
```

Verifying Your Setup

BASH

```
$ dbt debug

# Expected output:
# Configuration:
#   profiles.yml file [OK found and valid]
#   dbt_project.yml file [OK found and valid]
# Required dependencies:
#   git [OK found]
# Connection test:
#   Connection: OK
#
# All checks passed!
```

5

Building the Medallion Architecture (Models)

The Medallion Architecture organizes data transformations into three progressively refined layers. Each layer has a clear contract, making pipelines easier to debug and maintain.



Bronze Layer — Raw Ingestion

Define sources first in **sources.yml**, then reference them with `source()`:

YAML

```
# models/bronze/sources.yml
version: 2
sources:
  - name: raw_data
    database: dbt_tutorial_dev
    schema: source
    tables:
      - name: fact_sales
        description: "Raw transactional sales data from the source system"
      - name: fact_products
        description: Raw product catalog
```

SQL

```
-- models/bronze/bronze_sales.sql
-- {{ config(materialized='view') }}
select
    id            as sale_id,
    product_id,
    customer_id,
    sale_date,
    gross_amount,
    tax_amount,
    _loaded_at    as source_loaded_at
from {{ source('raw_data', 'fact_sales') }}
```

Silver Layer — Cleansed & Joined

SQL

```
-- models/silver/silver_sales_enriched.sql
-- {{ config(materialized='table') }}
select
    s.sale_id,
    s.sale_date,
    s.gross_amount,
    s.tax_amount,
    s.gross_amount + s.tax_amount as total_amount,
    p.category,
    p.product_name,
    p.unit_cost
from {{ ref('bronze_sales') }} s
left join {{ ref('bronze_products') }} p
    on s.product_id = p.id
where s.gross_amount is not null    -- Remove dirty rows
```

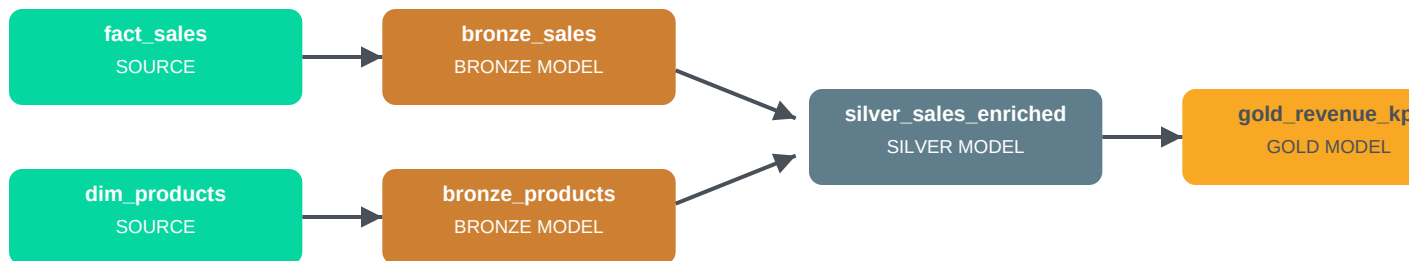
Gold Layer — Business-Ready

SQL

```
-- models/gold/gold_revenue_by_category.sql
-- {{ config(materialized='table') }}
select
  category,
  date_trunc('month', sale_date) as month,
  count(distinct sale_id)       as transaction_count,
  sum(total_amount)             as total_revenue,
  avg(total_amount)             as avg_order_value,
  sum(unit_cost * 1)            as estimated_cogs
from {{ ref('silver_sales_enriched') }}
group by 1, 2
```

The DAG — How ref() Builds the Lineage Graph

Every ref() and source() call creates a directed edge in the DAG. dbt resolves execution order automatically.



ref() / source() calls create directed edges — this IS the DAG

■ PRO TIP

Use `dbt ls --select +silver_sales_enriched` to see every upstream model that your Silver model depends on.

6

dbt Seeds: Handling Supplemental Static Data

Seeds are CSV files stored in the `seeds/` directory. dbt loads them directly into your warehouse as tables using `dbt seed`. They are ideal for lookup tables, mapping data, and reference sets that don't live in a source system.

When to Use Seeds vs. When to Avoid Them

■ Use Seeds For	■ Do NOT Use Seeds For
Country codes / ISO mappings	Frequently changing data
Category or status label lookups	Large datasets (> a few thousand rows)
Static cost / rate tables	Data that already exists in a source system
Department / team mappings	Data requiring complex transformations before load

Sample Seed & Configuration

CSV

```
-- seeds/mapping_categories.csv
id,category_name,is_active
1,Beverages,true
2,Electronics,true
3,Legacy Category,false
```

YAML

```
# Configure seed in dbt_project.yml
seeds:
  my_dbt_project:
    mapping_categories:
      +column_types:
        id: int
        is_active: boolean
      +schema: reference_data
```

SQL

```
-- Referencing a seed in a model
select
  s.sale_id,
  c.category_name
from {{ ref('silver_sales_enriched') }} s
left join {{ ref('mapping_categories') }} c -- ref() works for seeds!
  on s.category_id = c.id
where c.is_active = true
```

BASH

```
# CLI commands for seeds
dbt seed # Load all seeds
dbt seed --full-refresh # Drop and recreate (useful after schema change)
dbt seed --select mapping_categories # Load specific seed
```

7

Advanced Templating: Jinja & Macros

Jinja turns SQL into a dynamic programming environment. It allows you to follow the DRY principle — write logic once, reuse everywhere.

Jinja Fundamentals

Syntax	Purpose	Example
{{ expression }}	Output a value	{{ ref('my_model') }}
{% statement %}	Logic – if/for/set	{% if target.name == 'prod' %}
{# comment #}	Comments (not in compiled SQL)	{# TODO: add filter #}
{{ var('key') }}	Access project variable	{{ var('start_date') }}
{{ env_var('KEY') }}	Read environment variable	{{ env_var('DBT_TOKEN') }}
{{ this }}	Reference current model's table	where updated > (select max... from {{ this }})

The loop.last Pattern

Dynamically generating column lists creates a trailing comma problem. The loop.last check solves it elegantly:

SQL

```
select
  {% for col in ['sale_id', 'gross_amount', 'tax_amount', 'category'] %}
    {{ col }}{% if not loop.last %},{% endif %}
  {% endfor %}
from {{ ref('bronze_sales') }}

-- Compiles to:
-- select sale_id, gross_amount, tax_amount, category
-- from bronze.bronze_sales
```

Reusable Macros

SQL

```
-- macros/multiply.sql
{% macro multiply(column1, column2) %}
  ({{ column1 }} * {{ column2 }})
{% endmacro %}

-- Usage in a model:
select
  sale_id,
  {{ multiply('quantity', 'unit_price') }} as revenue
from {{ ref('bronze_sales') }}
```

SQL

```
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name, precision=2) %}
  round({{ column_name }} / 100.0, {{ precision }})
{% endmacro %}

-- Usage:
select {{ cents_to_dollars('amount_cents') }} as amount_dollars
from {{ ref('bronze_payments') }}
```

■ PRO TIP

Macros in the macros/ folder are automatically available to all models. No import needed. Name them clearly — they are your project's

8

Data Quality Framework (Testing)

dbt's testing framework ensures data integrity through two complementary strategies. Together they form a contract with your data: if a test fails, the pipeline should stop and alert.

Strategy 1: Generic Tests (schema-defined assertions)

Defined in `_models.yml` alongside your model definitions. Four built-in tests cover the most common quality checks:

Test	What It Checks	Common Use Case
unique	No duplicate values in column	Primary keys: sale_id, user_id
not_null	Column has no NULL values	Required fields
accepted_values	Column only contains listed values	status IN ('active','inactive','pending')
relationships	FK exists in referenced table	product_id references dim_products.id

YAML

```
# models/silver/_models.yml
version: 2
models:
  - name: silver_sales_enriched
    description: "Cleansed sales data joined with product info"
    columns:
      - name: sale_id
        description: "Primary key – must be unique"
        data_tests:
          - unique
          - not_null

      - name: category
        data_tests:
          - not_null
          - accepted_values:
              values: ['Beverages', 'Electronics', 'Apparel']

      - name: product_id
        data_tests:
          - relationships:
              to: ref('bronze_products')
              field: id
```

Strategy 2: Singular Tests (custom SQL assertions)

■ NEW IN THIS EDITION

The original PDF stated singular tests fail when rows are returned but did not explain WHY. This section explains the convention and

Convention: A singular test is a SQL query that returns rows only when something is *wrong*. An empty result = test passes. Any row returned = test fails. Think of it as: *'Find me evidence of a violation.'*

SQL

```
-- tests/assert_gross_amount_non_negative.sql
-- FAILS if any row has gross_amount < 0
-- Convention: returns rows only when the assertion is VIOLATED
select
  sale_id,
  gross_amount,
  'gross_amount is negative' as failure_reason
from {{ ref('silver_sales_enriched') }}
where gross_amount < 0
```

SQL

```
-- tests/assert_no_future_sales.sql
-- Business rule: sales cannot be dated in the future
select
  sale_id,
  sale_date,
  current_date() as today
from {{ ref('silver_sales_enriched') }}
where sale_date > current_date()
```

BASH

```
-- Run tests
dbt test                                # Run all tests
dbt test --select silver_sales_enriched # Test one model
dbt test --store-failures                # Save failing rows to DB
```

■ PRO TIP

Use --store-failures in CI/CD pipelines. Failed rows get written to a table in your warehouse so engineers can inspect exactly what b

9

Capturing History: Snapshots & SCD Type 2

Snapshots capture how rows change over time by maintaining **dbt_valid_from** and **dbt_valid_to** columns. This implements Slowly Changing Dimension Type 2 (SCD2) automatically.

The Deduplication Problem

Snapshots fail silently or produce incorrect history when the source table contains duplicate unique keys. Always deduplicate before snapshotting.

SQL

```
-- models/gold/intermediate/src_gold_items.sql
-- Dedup step: keep only the most recently updated row per id
with ranked as (
  select
    *,
    row_number() over (
      partition by id
      order by update_date desc
    ) as rn
  from {{ source('raw_data', 'items') }}
)
select * from ranked where rn = 1
```

Snapshot Configuration

YAML

```
# snapshots/gold_items_snapshot.yml
snapshots:
- name: gold_items_snapshot
  relation: "{{ ref('src_gold_items') }}" -- point to deduped model
  config:
    strategy: timestamp -- use 'check' for no updated_at column
    updated_at: update_date
    unique_key: id
    target_schema: snapshots
    invalidate_hard_deletes: true -- marks deleted rows as invalid
```

Resulting Snapshot Table Structure

id	name	status	dbt_valid_from	dbt_valid_to
101	Widget A	active	2024-01-01	2024-06-15
101	Widget A	discontinued	2024-06-15	null (current)
102	Gadget B	active	2024-01-01	null (current)

■ ■ **NOTE**

dbt_valid_to = null means the row is the CURRENT record. Filter WHERE dbt_valid_to IS NULL to get current state only.

BASH

```
dbt snapshot                # Run all snapshots
dbt snapshot --select gold_items_snapshot # Run specific
```

10 Documentation & Lineage

■ NEW IN THIS EDITION

This entire chapter is NEW — the original PDF mentioned the lineage graph but omitted the commands and workflow to generate a

dbt auto-generates a documentation site from your model YAML files, source definitions, and column descriptions. The lineage graph (DAG viewer) is arguably the most powerful feature for understanding and communicating your data pipeline.

The Documentation Workflow



All artifacts stored in `target/` folder. Commit `manifest.json` to Git for state-aware runs (`dbt build --state`).

Step 1 — Add Descriptions to Your YAML

```
# models/silver/_models.yml
version: 2
models:
  - name: silver_sales_enriched
    description: >
      Cleansed and enriched sales data. Joins bronze_sales with
      bronze_products to add category and cost information.
      Excludes rows where gross_amount is null.
    columns:
      - name: sale_id
        description: "Surrogate primary key for each sale transaction"
      - name: category
        description: "Product category from the products catalog"
      - name: total_amount
        description: "gross_amount + tax_amount. Used for revenue KPIs"
      - name: sale_date
        description: "Date the transaction was recorded in the source system"
```

YAML

Step 2 — Generate & Serve the Docs

BASH

```
# Generate catalog.json + manifest.json in target/
dbt docs generate

# Launch local documentation server (default: http://localhost:8080)
dbt docs serve

# Generate docs for specific models only
dbt docs generate --select silver+ # Silver and all downstream
```

What the Docs Site Shows

Section	What You'll Find
Overview	Project-level summary, source freshness, model counts
Model detail	SQL code, column descriptions, tests configured, tags
Lineage Graph (DAG)	Interactive visualization of all ref() and source() relationships
Source freshness	When each source table was last loaded (if configured)
Catalog	Column-level metadata pulled from the warehouse's information schema

Using manifest.json for State-Aware Runs

The manifest.json generated by dbt docs generate (or dbt parse) is also used for **state-aware CI/CD**.

You can run only models that changed since the last production run:

BASH

```
# In CI/CD: only run models that have changed
dbt build --select state:modified+ --state ./prod-artifacts/

# This requires copying the previous production manifest.json
# into a folder before running the comparison
```

■ PRO TIP

Commit your production manifest.json to a separate branch or artifact store. This enables 'slim CI' — only testing what changed, saving time and resources.

11

Orchestration, CI/CD & Deployment

Essential CLI Commands Reference

Command	What It Does	When to Use
<code>dbt debug</code>	Validates profiles, connection, project config	First thing after setup or errors
<code>dbt parse</code>	Compiles project, generates manifest (no SQL runs)	Before docs generate, in CI
<code>dbt run</code>	Executes models (creates tables/views)	Development & production
<code>dbt test</code>	Runs all configured tests	After dbt run
<code>dbt build</code>	seed + run + test + snapshot in DAG order	Preferred for CI/CD pipelines
<code>dbt seed</code>	Loads CSVs to warehouse	When seed data changes
<code>dbt snapshot</code>	Updates SCD2 snapshot tables	Scheduled daily jobs
<code>dbt docs generate</code>	Builds catalog.json + manifest.json	After any model change
<code>dbt docs serve</code>	Launches local docs server on :8080	For exploration & sharing
<code>dbt clean</code>	Deletes target/ and logs/ folders	Before fresh CI runs
<code>dbt ls</code>	Lists models matching a selection	Debugging selection syntax

Node Selection Syntax

Selector	Selects
<code>--select models/bronze+</code>	bronze models + ALL downstream dependents
<code>--select +silver_sales_enriched</code>	silver_sales_enriched + ALL upstream parents
<code>--select +silver_sales_enriched+</code>	Model + all parents + all children
<code>--select tag:daily</code>	All models tagged 'daily'
<code>--select source:raw_data</code>	All models that read from raw_data source
<code>--select state:modified+</code>	Changed models + their downstream (slim CI)
<code>--exclude models/gold</code>	Run everything EXCEPT gold models

The Schema Naming Problem in Production

■ NEW IN THIS EDITION

The original PDF showed the fix but not the problem. Here is the before/after.

■ Default (Dev prefix leaks to Prod)

```
bronze → default_bronze  
silver → default_silver  
gold   → default_gold
```

**■ With generate_schema_name macro**

```
bronze → bronze  
silver → silver  
gold   → gold
```

The generate_schema_name Override Macro

SQL

```
-- macros/generate_schema_name.sql  
-- Place this in macros/ - dbt finds it automatically  
{% macro generate_schema_name(custom_schema_name, node) -%}  
  {% set default_schema = target.schema -%}  
  
  {% if target.name == 'prod' and custom_schema_name is not none -%}  
    -- In prod: use EXACT custom schema (no username prefix)  
    {{ custom_schema_name | trim }}  
  
  {% else -%}  
    -- In dev: prefix with default_schema to avoid collisions  
    {{ default_schema }}_{{ custom_schema_name | trim if custom_schema_name else '' }}  
  
  {% endif -%}  
{%- endmacro %}
```

Full Production Deployment Flow

BASH

```
# Step 1: Clean previous artifacts to avoid state pollution
dbt clean

# Step 2: Validate everything before running
dbt debug --target prod

# Step 3: Run the full project in DAG order
# builds = seed + run + snapshot + test in correct order
dbt build --target prod

# Step 4: Regenerate docs with fresh catalog
dbt docs generate --target prod

# For CI/CD – only run what changed (slim CI)
dbt build --target prod --select state:modified+ --state ./prod-artifacts/
```

12

Quick Reference Card

■ NEW IN THIS EDITION

This reference card is NEW — a single-page cheat sheet covering the most-used patterns.

dbt Function Cheat Sheet

Function / Syntax	Returns / Does	Example
<code>ref('model_name')</code>	Dependency-aware table reference	<code>from {{ ref('bronze_sales') }}</code>
<code>source('src','table')</code>	Source table reference	<code>from {{ source('raw_data','fact') }}</code>
<code>var('key')</code>	Project variable value	<code>{{ var('start_date') }}</code>
<code>env_var('KEY')</code>	Environment variable value	<code>{{ env_var('DBT_TOKEN') }}</code>
<code>this</code>	Current model's relation	<code>WHERE updated > (SELECT MAX... FROM {{ this }})</code>
<code>config(materialized=)</code>	Set materialization inline	<code>{{ config(materialized='table') }}</code>
<code>is_incremental()</code>	True on incremental runs	<code>{% if is_incremental() %}</code>
<code>generate_schema_name()</code>	Override schema logic	See Chapter 11

Materialization Strategy Guide

Materialization	Creates	Best For	Avoid When
view	SQL View	Bronze / cheap transforms	Model queried very frequently
table	Physical Table	Silver / Gold models	Source data changes rarely
incremental	Append/merge new rows	Event logs, large fact tables	Small tables (overhead not worth it)
ephemeral	CTE (no DB object)	Intermediate helper queries	Model referenced in 5+ places
snapshot	SCD2 history table	Slowly Changing Dimensions	High-frequency updates

Most Common Troubleshooting Scenarios

Symptom	Likely Cause	Fix
dbt Power User: 'NoneType' error	Stale/missing manifest.json	Run dbt parse or dbt compile
Schema prefix in prod (default_bronze)	Missing generate_schema_name macro	Add macro from Chapter 11
Snapshot duplicates	Source has duplicate unique keys	Add dedup CTE before snapshot
Model runs but test fails	Data quality issue in source	Add singular test + --store-failures
Python 3.13 compile errors	dbt incompatible with 3.13	Downgrade to Python 3.12
profiles.yml not found	Wrong directory	Check ~/.dbt/ or set DBT_PROFILES_DIR

ref() circular dependency	Model A refs B, B refs A	Extract shared logic into ephemeral model
---------------------------	--------------------------	---

■ PRO TIP

Daily workflow: dbt debug → dbt build --select <your_model>+ → dbt test → dbt docs generate.
For production: dbt clean → dbt build --target prod → dbt docs generate --target prod.